PROYECTO DE CORTE 2 · FASE 2 HEXAGONAL

Bitácora de uso de IA

Juan Joya · Roger Bohórquez · Felipe Céspedes
Herramienta: Claude (Claude Code, app de escritorio)
Del 22 de septiembre al 29 de octubre de 2026

Qué le pedimos. Primero, revisar el estado del repositorio contra cada enunciado y preguntarnos las decisiones, con opciones y una recomendada. Después, un plan por fases, y construir el código, las pruebas, el README, los diagramas, el firmware de la ESP32 y el simulador, con un commit por fase. Al final, revisar todo con varios revisores y probar con la placa real. Las decisiones de la tabla las tomamos nosotros; el código, las pruebas, los diagramas del repositorio y el README los escribió la IA a partir de esas decisiones. **El documento de arquitectura lo escribimos nosotros:** la IA solo lo comparó con el enunciado y con el código, y nos dijo qué faltaba.

Qué propuso o hizo la IA	Qué se decidió	Por qué
Para las cuentas: una base por usuario, o una sola base con el dueño en cada fila.	Una sola base SQLite, con el dueño en cada fila.	Es más simple de migrar y de consultar para el administrador.
Poner la regla de permisos en el dominio (`Usuario.exigir_admin()`) y no en cada ruta.	Aceptado, más una regla: un admin no puede actuar sobre su propia cuenta.	La regla vale igual por HTTP, por consola o por un canal futuro, y siempre queda un admin.
Sesiones con token opaco, guardando su huella SHA-256 (corte 2).	**Lo cambiamos en la fase 2:** JWT firmados con PyJWT y revocables, porque el servidor guarda su `jti`.	R7 pide JWT, y no queríamos perder el cierre de sesión ni la desactivación inmediata.
Pasar las especies a SQLite y eliminar el CSV (corte 2).	**En la fase 2 volvió el CSV, con una idea nuestra:** es una copia de la base, de solo lectura, y se elige por configuración.	R4 pide CSV o base de datos. Con una copia, las dos tablas no pueden contradecirse.
Al revisar el código contra la fase 2, encontró que no había puertos de entrada, que la propiedad la decidía el SQL y que los `Repositorio*` eran grandes.	Puertos de entrada como clases abstractas, con el nombre del caso de uso; puertos pequeños; la propiedad en el dominio (`Planta.exigir_dueno`).	Así lo pide la arquitectura hexagonal. Una planta ajena da 404, igual que una inexistente, para no revelar que existe.
Varias formas de autenticar al dispositivo.	La matera nace con una clave; la placa la canjea por un JWT de alcance «lecturas» y manda cada lectura con su hora (NTP).	Alcances distintos para usuario y dispositivo (R7), y una hora en el futuro se puede rechazar.
Un simulador del ESP32 para la demo.	**Lo cambiamos:** la demo es con la ESP32 real, y el simulador queda como respaldo y para los casos de error (`--casos`).	Teníamos la placa. El simulador cubre R3 sin depender del hardware.
Adaptó el firmware a los sensores que de verdad tenía la placa: DHT11 y un LDR digital, no los que estaban planeados.	Se manda la humedad del aire, y lo decimos en la presentación. La luz va en lux: 1500 con luz y 50 sin luz.	El dominio no cambia, y la limitación queda declarada.
Una revisión final con cuatro revisores (seguridad, enunciado, documentación y clientes). Nada descalificante, pero Diagnóstico y Especies seguían públicos.	Todas las rutas del front exigen el JWT de usuario. Además: los sensores de una cuenta desactivada no reportan, y en modo CSV el backend no arranca si la copia no es igual a la base.	El enunciado dice que los endpoints del front exigen un JWT de usuario. Lo cumplimos al pie de la letra.
Revisó nuestro borrador del documento de arquitectura contra el enunciado y el código.	Corregimos lo que señaló: 5 páginas, faltaba el enlace al commit de la prueba de cambio y el diagrama tenía flechas y nombres que no coincidían.	El documento es nuestro; la revisión solo comprobó que dijera lo mismo que el código.

Problemas que aparecieron y cómo se corrigieron

- Con una sola conexión SQLite, el servidor real fallaba aunque las pruebas pasaran, porque Flask atiende cada petición en un hilo. Se abre una conexión por operación, y una prueba levanta el servidor real.
- El historial se ordenaba mal cuando el dispositivo mandaba la hora con otra zona horaria. Ahora se guarda en UTC.
- Google Drive volvió a bajar un archivo con una falla metida a propósito para una prueba. Se restauró con Git, y desde entonces esas pruebas corren sobre una copia fuera de Drive.
- Un script de la IA dejó un archivo vacío a mitad de una edición. Se restauró de Git sin pérdida, y los scripts ahora revisan el texto antes de escribir.
- La placa no llegaba al PC en una red con bandas de 2,4 y 5 GHz hasta que el PC le hacía ping. Para la demo usamos el hotspot del celular, donde funcionó a la primera.
- La IA nos dio `python main.py`, que no funciona en nuestro PC porque Python no está en el PATH. Se corrigió con la ruta completa.

Cómo se comprobó que la arquitectura hace lo que dice

- 443 pruebas en verde. Las 167 de dominio y aplicación siguen pasando en una copia sin la carpeta `infraestructura/`.
- `git show --stat d41ca34 -- backend/dominio backend/aplicacion` sale vacío: la prueba de cambio no tocó el núcleo.
- Cada regla de seguridad se probó con mutantes: se quita la regla a propósito, en una copia, y una prueba tiene que fallar.
- El simulador en modo `--casos` responde «12 de 12 como se esperaba» contra el backend.
- Con la ESP32 en el hotspot del celular, el canje y una lectura cada 15 segundos llegaron con 201 y quedaron en el historial.

Usamos IA para escribir código, depurar y explorar ideas, como permite el enunciado. Revisamos lo que entregamos y lo respaldamos.